UNIT- 4

Association Analysis: Problem Definition, Frequent Itemset Generation, Rule Generation: Confident Based Pruning, Rule Generation in Apriori Algorithm, Compact Representation of frequent item sets, FP-Growth Algorithm.

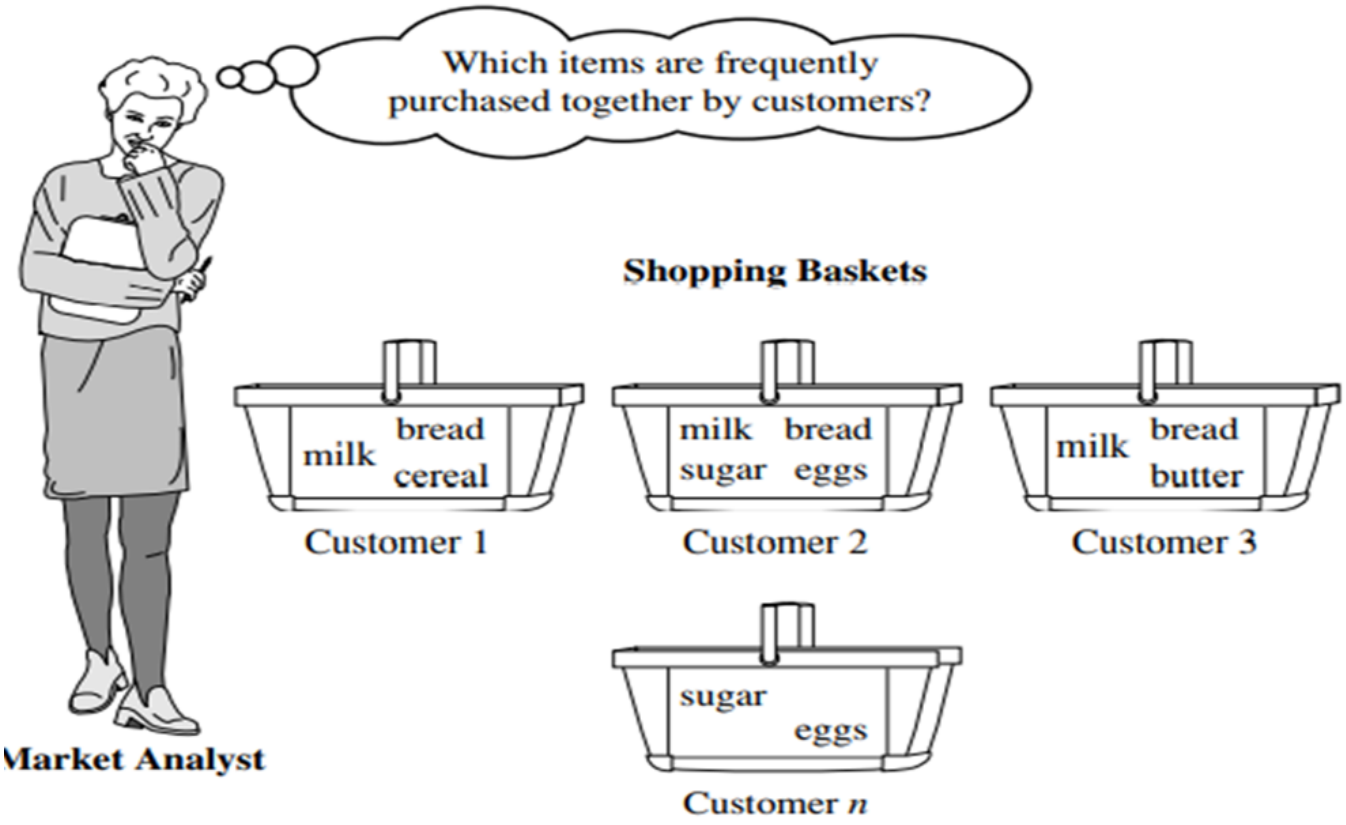
4.1 PROBLEM DEFINITION

- Association Rule Mining discovers interesting relationships in large datasets.
- Used in market basket analysis, catalog design, and customer behavior understanding.
- Finds associations, correlations, or patterns among sets of items.

Frequent Itemset Mining / Association Rule Mining / Market Basket Analysis:

- Frequent itemset mining leads to the discovery of associations and correlations among items in large transactional data sets.
- The discovery of interesting correlation relationships among huge amounts of business transaction records can help in many business decision-making processes.
- A typical example of frequent itemset mining is market basket analysis.
- This process analyzes customer buying habits by finding associations between the different items that customers place in their "shopping baskets"

Market basket analysis



- Analyze the buying patterns that reflect items that are frequently purchased together.
- These patterns can be represented in the form of Association rules.
- **Rule form: "A => B" [support, confidence]**

- For example, the information that customers who purchase computers Also tend to buy software at the same time is represented as:

Computer=>financial_management_software[support=2%,confidence= 60%]

- Typically, association rules are considered interesting if they satisfy both a **minimum support threshold** and a **minimum confidence threshold** such thresholds can be set by the users or domain experts.

Frequent Itemsets, Closed Itemsets, and Association Rules:

- Let I be a set of items {I1, I2, I3,…, Im},Let D be a set of database transactions where each transaction T is a set of items such that $T \subseteq I$.
- Each transaction is associated with an identifier , called TID.
- Let A be a set of items. A transaction T is said to contain A if and only if $A \subseteq T$.
- An a**ssociation rule** is an implication of the form

$A \Rightarrow B$

where $A \subseteq I$, $B \subseteq I$, and $A \cap B = \Phi$.

- The rule A=> B holds in the transaction set D with **support** s, where s is the percentage of transactions in D that contain $A \cup B$ (i.e both A and B). This is taken to be the probability, $P(A \cup B)$.
- The rule A=>B has **confidence** c in the transaction set D if c is the percentage of transactions in D containing A that also contain B. This is taken to be the conditional probability P(B/A).
- That is Support (A=>B) = $P(A \cup B)$.
- Confidence (A=>B) = P(B/A).
- Support says that 67% of customers purchased milk and cheese.
- Confidence is that 100% of the customers that bought milk also bought cheese
- **Strong Association Rules :** Rules that satisfy both a minimum support threshold (min_sup) and a minimum confidence threshold (min_conf) are called strong.
- By convention, we write support and confidence value so as to occur between 0% and 100% rather than 0 to 1.0.

Item set : A set of items is referred to as an itemset. An itemset that contains k items is a k-itemset.

- Ex : The set{computer, financial_management_software} is a 2-itemset.

Support count : The frequency of an itemset is the number of transactions that contain the itemset.

- This is also known as the frequency, support count or count of the itemset.

Frequent itemset: if an itemset satisfies minimum support, then it is a frequent itemset .

- The set of frequent K-itemsets is commonly denoted by Lk.

In general, association rule mining can be viewed as a two-step process:

1. **Find all frequent itemsets:** By definition, each of these itemsets should satisfy minimum support count, min sup.

2. **Generate strong association rules from the frequent itemsets**: By definition, these rules must satisfy minimum support and minimum confidence

Support & confidence:

- Support(s) of an association rule is defined as the percentage of records that contain A U B to the total number of records in the database.

Support (A->B)=P (A U B) = support_count(A U B)/count(all_transactions)

- Confidence of an association rule is defined as the percentage of the number of transactions that contain A U B to the total number of records that contain A.

Confidence(A-> B) =P(B/A) =support(AUB)/support(A).=support_count(AU B)/support_count(A).

Example : Data set D

TID	Itemsets
T100	1 3 4
T200	2 3 5
T300	1 2 3 5
T400	2 5

Support count, Support and Confidence:

Support count(1,3)=2

|D| = 4

Support(1->3)=2/4 = 0.5

Support(3->2)=2/4 =0.5

Confidence(3->2)= Support Count(2 U 3) / Support count(3)

= 2/3

= 0.67

4.2 FREQUENT ITEMSET GENERATION

1. Itemset Lattice

- If you have k items, you can form up to 2^k – 1 itemsets (excluding the empty set).
- Example: For {a, b, c, d, e}, all possible subsets form a lattice structure (Figure 5.1).
- The lattice is basically a map of all possible item combinations.

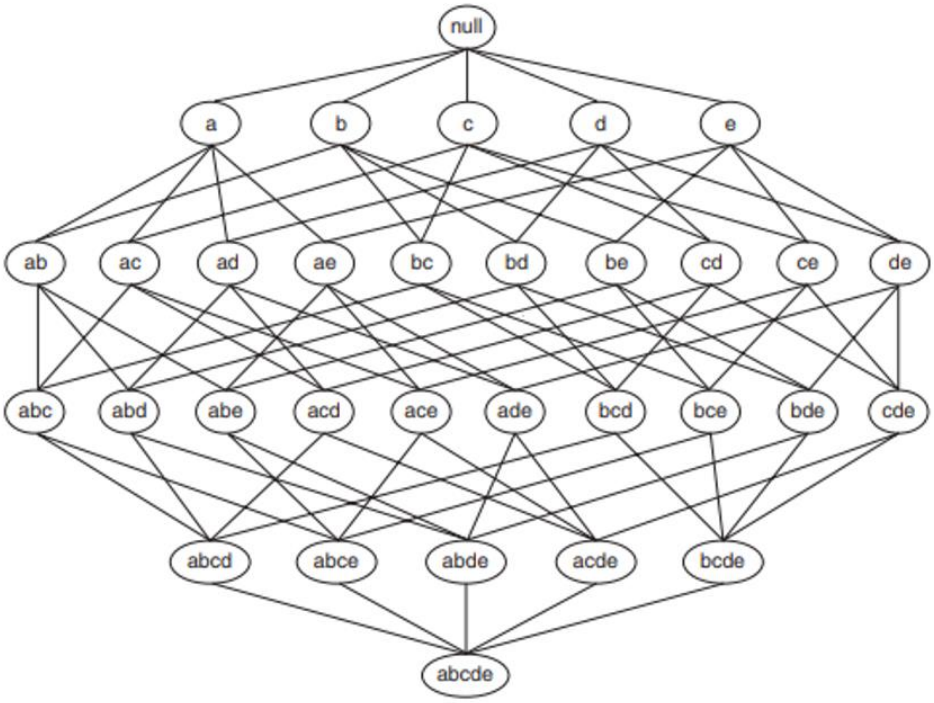


Figure 5.1. An itemset lattice.

2. Brute Force Problem

- To find frequent itemsets, you check every candidate against every transaction.
- Support of an itemset = how many transactions contain it
- But brute force needs O(N × M × w) comparisons:

N = number of transactions

M = number of candidate itemsets (2^k – 1)

w = max transaction width

- This becomes very expensive when k is large.

3.Three Ways to Reduce Computation

[i] Reduce number of candidate itemsets (M):

- Use Apriori principle → If an itemset is infrequent, all its supersets are also infrequent.

[ii] Reduce comparisons:

- Use better data structures (hash trees, FP-trees) or compress the dataset.

[iii] Reduce number of transactions (N):

- As itemsets get larger, fewer transactions support them → prune transactions that can't contribute.

4.2.1 The Apriori Principle

- Finding Frequent Itemsets by Confined Candidate Generation
- The name of the algorithm is based on the fact that the algorithm uses **prior knowledge** of frequent itemset properties

- Uses an iterative approach known as a level-wise search, where k itemsets are used to explore (k+1)-itemsets.
- First, the set of frequent 1-itemsets is found.
- This set is denoted by L1. L1 is used to find L2 , the set of frequent 2-itemsets,which is used to find L3 ,and so on, until no more frequent k-itemsets can be found.
- The finding of each Lk requires one full scan of the database.

Apriori property: All subsets of a frequent itemset must also be frequent.

The Apriori property is based on

- if an itemset I does not satisfy the min sup, threshold, then I is not frequent
- If an item A is added to the itemset I, then the resulting itemset cannot occur more frequently than I. Therefore, this is not frequent.
- This property belongs to a special category of properties called **antimonotonicity.**

Anti-Monotone property –

- Using the apriori property in the algorithm:
- Let us look at how Lk-1 is used to find Lk, for k>=2
- Two steps:

Join : Ck is generated by joining Lk-1with itself.

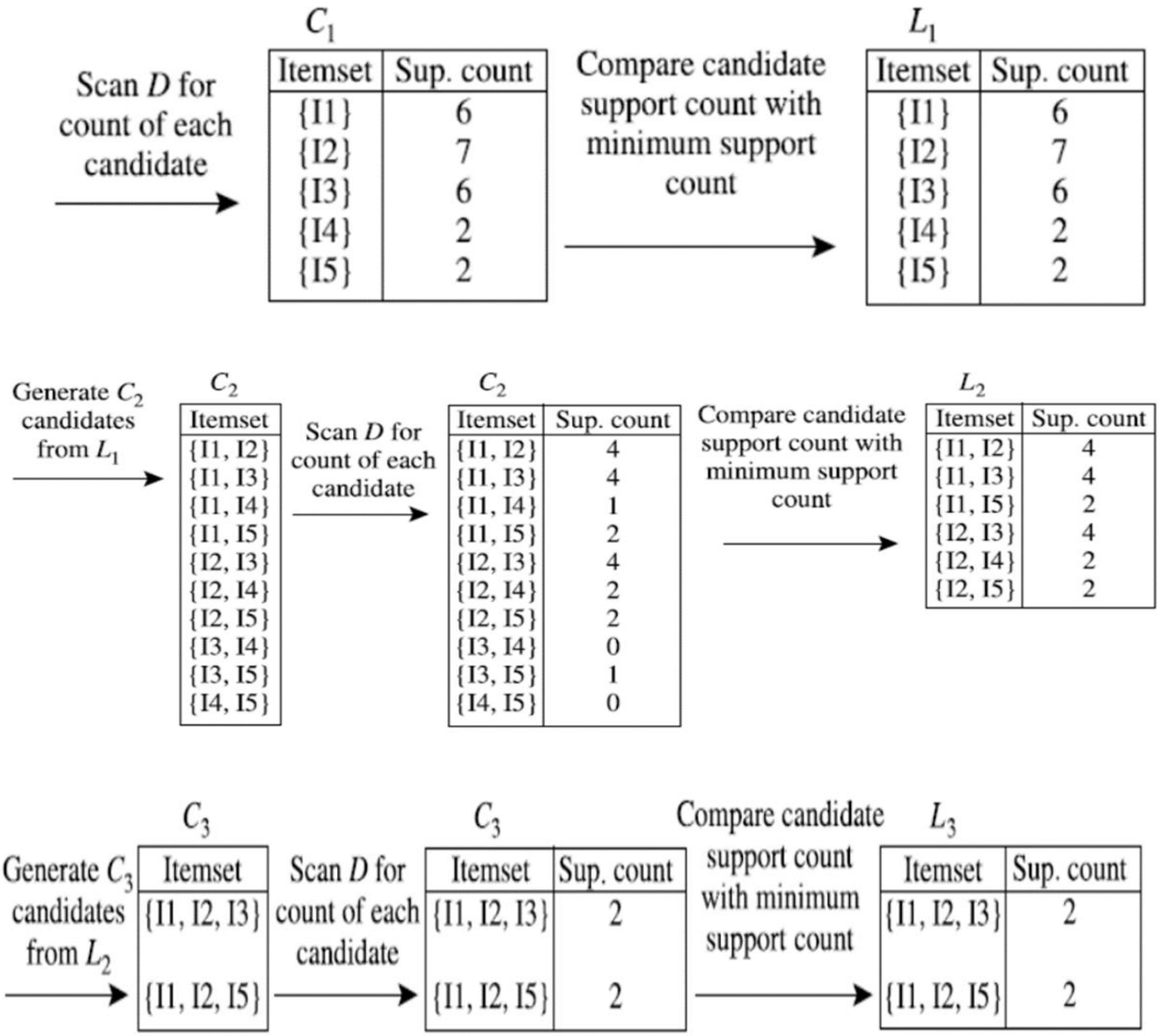
Prune: Any (k-1)-itemset that is not frequent cannot be a subset of a frequent k-itemset.

- Scaning the database to determine the count of each candidate in Ck
- To reduce the size of Ck the Apriori property is used: if any (k-1) subset of a candidate k-itemset is not in Lk-1, then the candidate cannot be frequent ,so it can be removed from Ck.

Example: Transactional data for an All Electronics branch

TID	List of items
T100	I1,I2,I5
T200	I2,I4
T300	I2,I3
T400	I1,I2,I4
T500	I1,I3
T600	I2,I3
T700	I1,I3
T800	I1,I2,I3,I5
T900	I1,I2,I3

- Generation of the candidate itemsets and frequent itemsets, where the minimum **support count is 2.**



- The above figure: represents Generation of candidate itemsets and frequent itemsets, where the minimum support count is 2.
- Scan D for count of each candidate

  C1: I1 – 6, I2 – 7, I3 -6, I4 – 2, I5 - 2

Compare candidate support count with minimum support count (min_sup=2)

- L1: I1 – 6, I2 – 7, I3 -6, I4 – 2, I5 - 2
- Generate C2 candidates from L1 and scan D for count of each candidate
- C2: {I1,I2} – 4, {I1, I3} – 4, {I1, I4} – 1, …
- Compare candidate support count with minimum support count
- L2: {I1,I2} – 4, {I1, I3} – 4, {I1, I5} – 2, {I2, I3} – 4, {I2, I4} - 2, {I2, I5} – 2

Generate C3 candidates from L2 using the join and prune steps:

- Join: C3=L2xL2={{I1, I2, I3}, {I1, I2, I5}, {I1, I3, I5}, {I2, I3, I4}, {I2, I3, I5}, {I2, I4, I5}}

- Prune: C3: {I1, I2, I3}, {I1, I2, I5}
- Scan D for count of each candidate

C3: {I1, I2, I3} - 2, {I1, I2, I5} – 2

Compare candidate support count with minimum support count

- L3: {I1, I2, I3} – 2, {I1, I2, I5} – 2
- Generate C4 candidates from L3
- C4=L3xL3={I1, I2, I3, I5}
- This itemset is pruned, because its subset {{I2, I3, I5}} is not frequent => C4=null

Algorithm: Apriori. Find frequent itemsets using an iterative level-wise approach based on candidate generation.

Input:

- *D*, a database of transactions;
- *min_sup*, the minimum support count threshold.

Output: *L*, frequent itemsets in *D*.

Method:

```
(1)  L1 = find_frequent_1-itemsets(D);
(2)  for (k = 2; Lk−1 ≠ φ; k++) {
(3)      Ck = apriori_gen(Lk−1);
(4)      for each transaction t ∈ D { // scan D for counts
(5)          Ct = subset(Ck, t); // get the subsets of t that are candidates
(6)          for each candidate c ∈ Ct
(7)              c.count++;
(8)      }
(9)      Lk = {c ∈ Ck | c.count ≥ min_sup}
(10) }
(11) return L = ∪k Lk;
```

```
procedure apriori_gen(L_{k-1}:frequent (k-1)-itemsets)
(1)     for each itemset l_1 ∈ L_{k-1}
(2)         for each itemset l_2 ∈ L_{k-1}
(3)             if (l_1[1] = l_2[1]) ∧ (l_1[2] = l_2[2])
                    ∧... ∧ (l_1[k-2] = l_2[k-2]) ∧ (l_1[k-1] < l_2[k-1]) then {
(4)                 c = l_1 ⋈ l_2; // join step: generate candidates
(5)                 if has_infrequent_subset(c, L_{k-1}) then
(6)                     delete c; // prune step: remove unfruitful candidate
(7)                 else add c to C_k;
(8)             }
(9)     return C_k;

procedure has_infrequent_subset(c: candidate k-itemset;
          L_{k-1}: frequent (k-1)-itemsets); // use prior knowledge
(1)     for each (k-1)-subset s of c
(2)         if s ∉ L_{k-1} then
(3)             return TRUE;
(4)     return FALSE;
```

- **Support 30% confidence 60%**

Transactions List

1	Milk	Egg	Bread	Butter
2	Milk	Butter	Egg	Ketchup
3	Bread	Butter	Ketchup	
4	Milk	Bread	Butter	
5	Bread	Butter	Cookies	
6	Milk	Bread	Butter	Cookies
7	Milk	Cookies		
8	Milk	Bread	Butter	
9	Bread	Butter	Egg	Cookies
10	Milk	Butter	Bread	
11	Milk	Bread	Butter	
12	Milk	Bread	Cookies	Ketchup

Transactions List

1	Milk	Egg	Bread	Butter
2	Milk	Butter	Egg	Ketchup
3	Bread	Butter	Ketchup	
4	Milk	Bread	Butter	
5	Bread	Butter	Cookies	
6	Milk	Bread	Butter	Cookies
7	Milk	Cookies		
8	Milk	Bread	Butter	
9	Bread	Butter	Egg	Cookies
10	Milk	Butter	Bread	
11	Milk	Bread	Butter	
12	Milk	Bread	Cookies	Ketchup

1-item Sets	Frequency
Milk	9
Bread	10
Butter	10
Egg	3
Ketchup	3
Cookies	5

Frequent 1-item Sets	Frequency
Milk	9
Bread	10
Butter	10
Cookies	5

Transactions List

1	Milk	Egg	Bread	Butter
2	Milk	Butter	Egg	Ketchup
3	Bread	Butter	Ketchup	
4	Milk	Bread	Butter	
5	Bread	Butter	Cookies	
6	Milk	Bread	Butter	Cookies
7	Milk	Cookies		
8	Milk	Bread	Butter	
9	Bread	Butter	Egg	Cookies
10	Milk	Butter	Bread	
11	Milk	Bread	Butter	
12	Milk	Bread	Cookies	Ketchup

2-item Sets	Frequency
Milk, Bread	7
Milk, Butter	7
Milk, Cookies	3
Bread, Butter	9
Butter, Cookies	3
Bread, Cookies	4

Frequent 2-item Sets	Frequency
Milk, Bread	7
Milk, Butter	7
Bread, Butter	9
Bread, Cookies	4

Transactions List

1	Milk	Egg	Bread	Butter
2	Milk	Butter	Egg	Ketchup
3	Bread	Butter	Ketchup	
4	Milk	Bread	Butter	
5	Bread	Butter	Cookies	
6	Milk	Bread	Butter	Cookies
7	Milk	Cookies		
8	Milk	Bread	Butter	
9	Bread	Butter	Egg	Cookies
10	Milk	Butter	Bread	
11	Milk	Bread	Butter	
12	Milk	Bread	Cookies	Ketchup

Milk, Bread, Butter, Cookies

3-item Sets	Frequency
Milk, Bread, Butter	6
Milk, Bread, Cookies	1
Bread, Butter, Cookies	3
Milk, Butter, Cookies	2

Frequent 3-item Sets	Frequency
Milk, Bread, Butter	6

4.2.2 Frequent Itemset Generation in the Apriori Algorithm

- The Apriori algorithm aims to find frequent itemsets efficiently. Using the Apriori principle, it prunes all supersets of an infrequent itemset, thus reducing unnecessary computations.

1.Start with 1-itemsets (C1).

- Count their supports → keep frequent ones (F1).

2.Generate 2-itemsets (C2) from F1.

- Count supports → keep frequent ones (F2).

3.Generate 3-itemsets (C3) from F2.

- Keep only those whose all subsets are frequent → prune others.

4.Repeat until no more frequent itemsets.

- Algorithm stops when Fk = ∅.

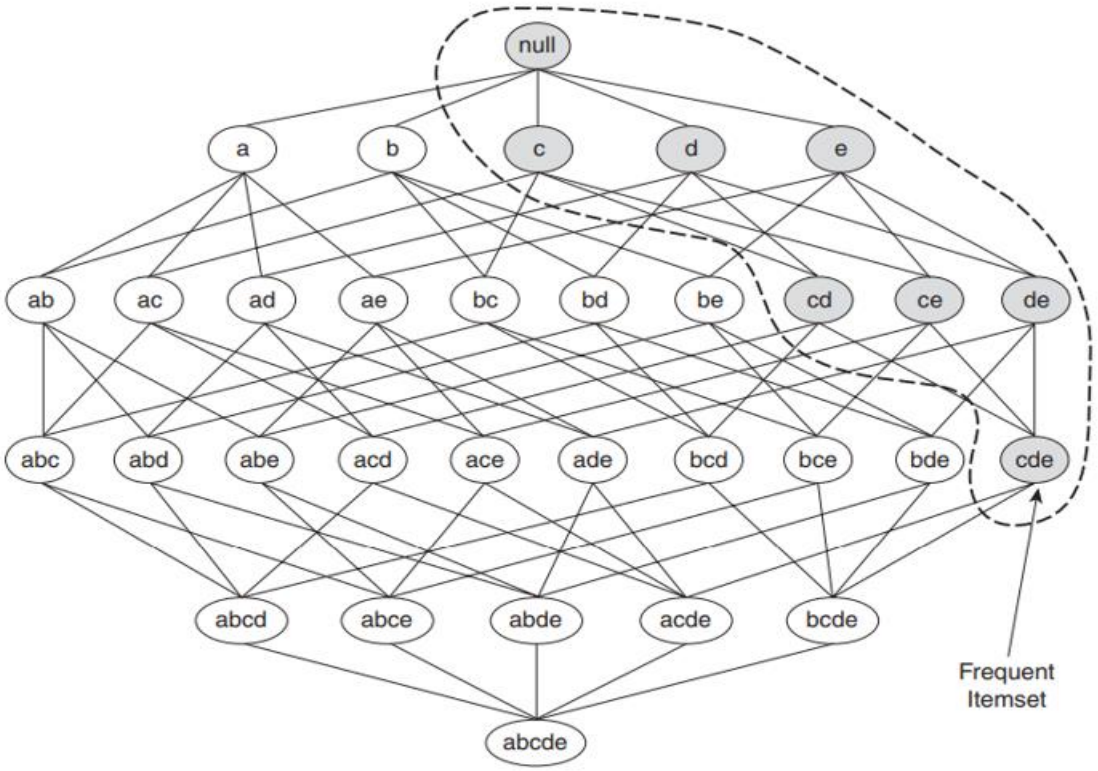


Figure 5.3. An illustration of the *Apriori* principle. If $\{c, d, e\}$ is frequent, then all subsets of this itemset are frequent.

- This helps in building larger frequent itemsets from smaller ones.

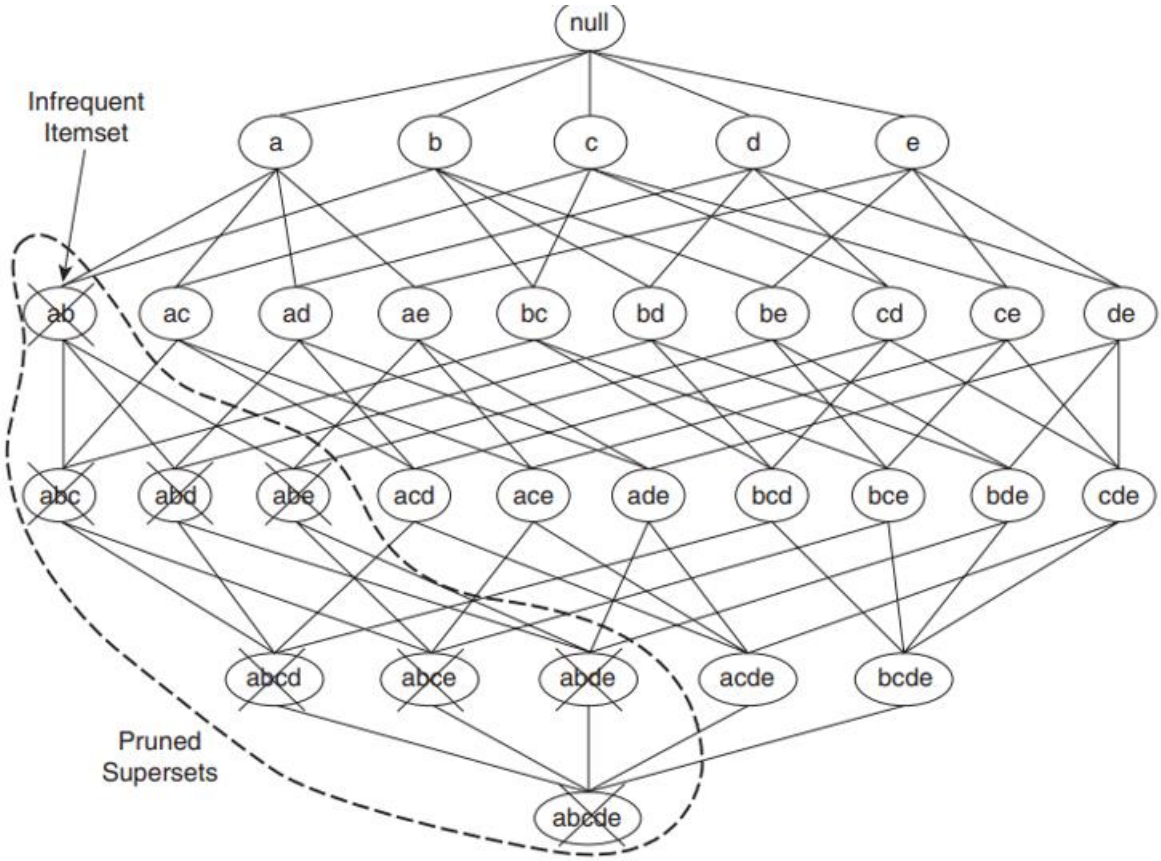


Figure 5.4. An illustration of support-based pruning. If $\{a, b\}$ is infrequent, then all supersets of $\{a, b\}$ are infrequent.

- This allows pruning the search space early, saving computation

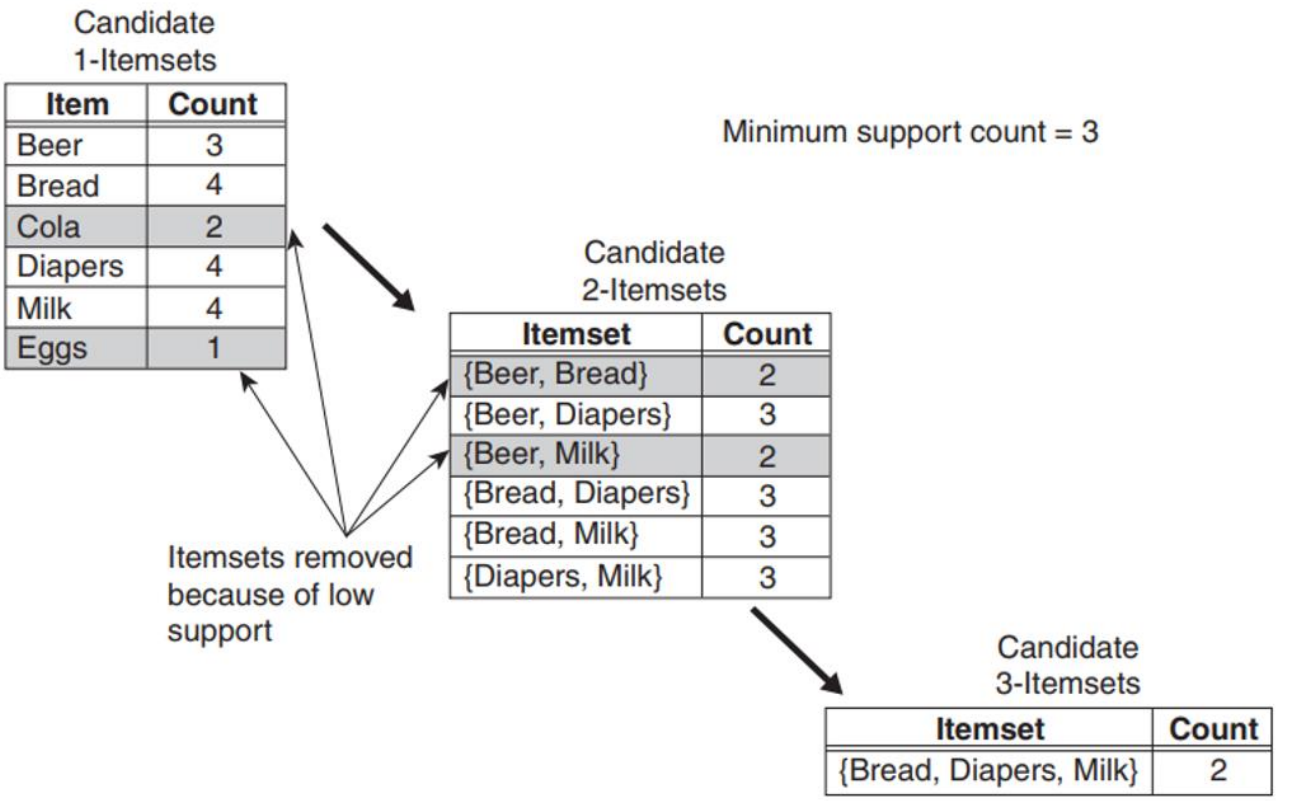


Figure 5.5. Illustration of frequent itemset generation using the Apriori algorithm.

4.3 RULE GENERATION

4.3.1 Confident Based Pruning

- After generating association rules from frequent itemsets, not all rules are useful.
- Confidence measures the reliability of a rule:

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

- If the confidence is below the minimum threshold, the rule is pruned (discarded).
  Rule: {Bread} → {Milk}
- Suppose support(Bread, Milk) = 3, support(Bread) = 4
- Confidence = 3/4 = 75%
- If minsup = 60% and minconf = 70% → Rule is kept.
- If confidence < 70% → pruned.

4.3.2 Rule Generation in Apriori Algorithm

Generating association rules from frequent itemsets :

- Generating strong association rules:
- For each frequent itemset "l", generate all nonempty subsets of l.
- Confidence(A=>B)=P(B|A)=support_count(AUB)/support_count(A)
- support_count(AUB) – number of transactions containing the itemsets AUB

- support_count(A) - number of transactions containing the itemsets A
- **Generating association rules.** Let's try an example based on the transactional data for AllElectronics
- The data contain frequent itemset X ={I1, I2,I5}.
- What are the association rules that can be generated from X?
- The nonempty subsets of X are {I1, I2}, {I1, I5}, {I2, I5}, {I1}, {I2}, and {I5}.

$$\{I1, I2\} \Rightarrow I5, \quad confidence = 2/4 = 50\%$$
$$\{I1, I5\} \Rightarrow I2, \quad confidence = 2/2 = 100\%$$
$$\{I2, I5\} \Rightarrow I1, \quad confidence = 2/2 = 100\%$$
$$I1 \Rightarrow \{I2, I5\}, \quad confidence = 2/6 = 33\%$$
$$I2 \Rightarrow \{I1, I5\}, \quad confidence = 2/7 = 29\%$$
$$I5 \Rightarrow \{I1, I2\}, \quad confidence = 2/2 = 100\%$$

- If the minimum confidence threshold is, 70%, then only the second, third, and last rules are strong.

- Non-Empty subset are
  - {{Milk}, {Bread}, {Butter}, {Milk, Bread}, {Milk, Butter}, {Bread, Butter}}
  - Min_Support = 30% and Min_Confidence = 60%

- Rule 3: {Butter} → {Milk, Bread} {S=50%, C=60%}
  - Support = 6/12 = 50%
  - Confidence = Support (Milk, Bread, Butter)/Support(Butter) = 6/10 = 60%>=60
  - Valid

- Rule 4: {Milk, Bread} → {Butter} {S=50%, C=85.7%}
  - Support = 6/12 = 50%
  - Confidence = Support (Milk, Bread, Butter)/Support(Milk, Bread) = 6/7 = 85.7% > 60%
  - Valid

- Non-Empty subset are
  - {{Milk}, {Bread}, {Butter}, {Milk, Bread}, {Milk, Butter}, {Bread, Butter}}
  - Min_Support = 30% and Min_Confidence = 60%

- Rule 5: {Milk, Butter} → {Bread} {S=50%, C=85.7%}
  - Support = 6/12 = 50%
  - Confidence = Support (Milk, Bread, Butter)/Support(Milk, Butter) = 6/7 = 85.7% >= 60%
  - Valid

- Rule 6: {Bread, Butter} → {Milk} {S=50%, C=66.67%}
  - Support = 6/12 = 50%
  - Confidence = Support (Milk, Bread, Butter)/Support(Bread, Butter) = 6/9 = 66.67%>=60
  - Valid

Drawbacks of Apriori

- It still need to generate a huge number of candidate sets.
- It need to repeatedly scan the whole database.

4.4 COMPACT REPRESENTATION OF FREQUENT ITEM SETS

- 4.4.1 Maximal Frequent Itemsets
- 4.4.2 Closed Frequent Itemsets
  - **Closed Itemset:** An itemset is closed if none of its immediate supersets have same support count same as Itemset.
  - **A maximal frequent** itemset is represented as a frequent itemset for which none of its direct supersets are frequent.

TID	List of Items
T1	A, B, C, D
T2	A, B,C, D
T3	A,B,C
T4	B, C, D
T5	C,D

Min Support Count=3,

Find Frequent, Closed, Maximal Itemset

Item	Count
A	3
B	4
C	5
D	4

All items that is A, B, C & D are **frequent** because they sup count is greater than or equal to minimum support count.

Item	Count
A, B	3
A, C	3
A, D	2
B, C	4
B, D	3
C, D	4

All 2 items sets are frequent, except AD

A(3) → AB(3) , AC(3), AD(2)

A(count) is not greater than its immediate superset.

A is not closed.

In A's immediate superset, itemset are present with min. support count i.e. 3.

A is not maximal

B(4) → AB(3) , BC(4), BD(3)

B(count) is not greater than its immediate superset.

B is not closed.

In B's immediate superset, itemset are present with min. support count i.e. 3.

B is not maximal

Item	Count
A, B	3
A, C	3
A, D	2
B, C	4
B, D	3
C, D	4

C(5) → AC(3) , BC(4), CD(2)

C(count) is greater than its immediate superset.

C is closed.

In C's immediate superset, itemset are present with min. support count i.e. 3.

C is not maximal

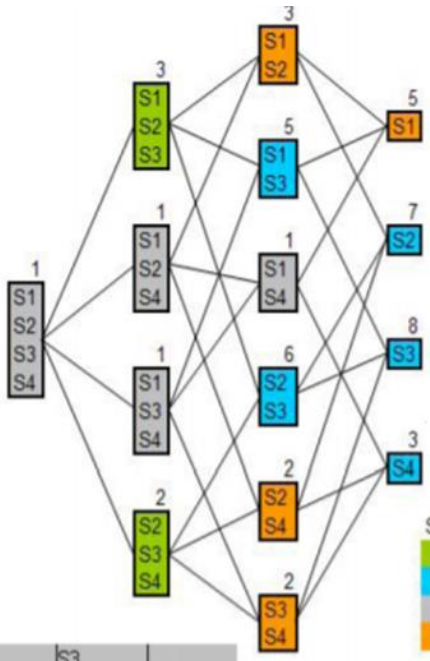
D(4) → AD(2) , BD(3), CD(4)

D(count) is not greater than its immediate superset.

D is not closed.

In D's immediate superset, itemset are present with min. support count i.e. 3.

D is not maximal



- A frequent itemset is one that occurs in at least a user-specific percentage of the database. That percentage is called support.
- An itemset is closed if none of its immediate supersets has the same support as the itemset.
- An itemset is maximal frequent if none of its immediate supersets is frequent.

Support = 2
Maximal Frequent Itemset
Closed Frequent Itemset
Non-Frequent Itemset
Non-Closed Frequent Itemsets

C1	S1		S3	
C2		S2		
C3				S4
C4		S2	S3	S4
C5		S2	S3	
C6		S2	S3	
C7	S1	S2	S3	S4
C8	S1		S3	
C9	S1	S2	S3	
C10	S1	S2	S3	

4.5 FP-GROWTH ALGORITHM

4.5.1 FP-Tree Representation

A Pattern-Growth Approach for Mining Frequent Itemsets

Two Steps:

- Scan the transaction DB for the **first time**, find frequent items (single item) and order them into a list L in descending order.
- In the format of **(item-name, support)**
- For each transaction, order its frequent items according to the order in L; **Scan DB the second time**, construct FP-tree by putting each frequency ordered transaction onto it.

Example:

TID	List of items
T100	I1,I2,I5
T200	I2,I4
T300	I2,I3
T400	I1,I2,I4
T500	I1,I3
T600	I2,I3
T700	I1,I3
T800	I1,I2,I3,I5
T900	I1,I2,I3

- The first scan of the database is the same as Apriori, which derives the set of frequent (1-itemsets) and their support counts.
- Let the **minimum support count be 2.**
- The set of frequent items is sorted in the order of descending support count.
- This resulting set or list is denoted by L.

Example

TID	List of items
T100	I1,I2,I5
T200	I2,I4
T300	I2,I3
T400	I1,I2,I4
T500	I1,I3
T600	I2,I3
T700	I1,I3
T800	I1,I2,I3,I5
T900	I1,I2,I3

- I2,I1,I5
- I2,I4
- I2,I3
- I2,I1,I4
- I1,I3
- I2,I3
- I1,I3
- I2,I1,I3,I5
- I2,I1,I3

Items	Sup.count
I2	7
I1	6
I3	6
I4	2
I5	2

FP-Tree Definition :

FP-tree is a **frequent pattern tree.**

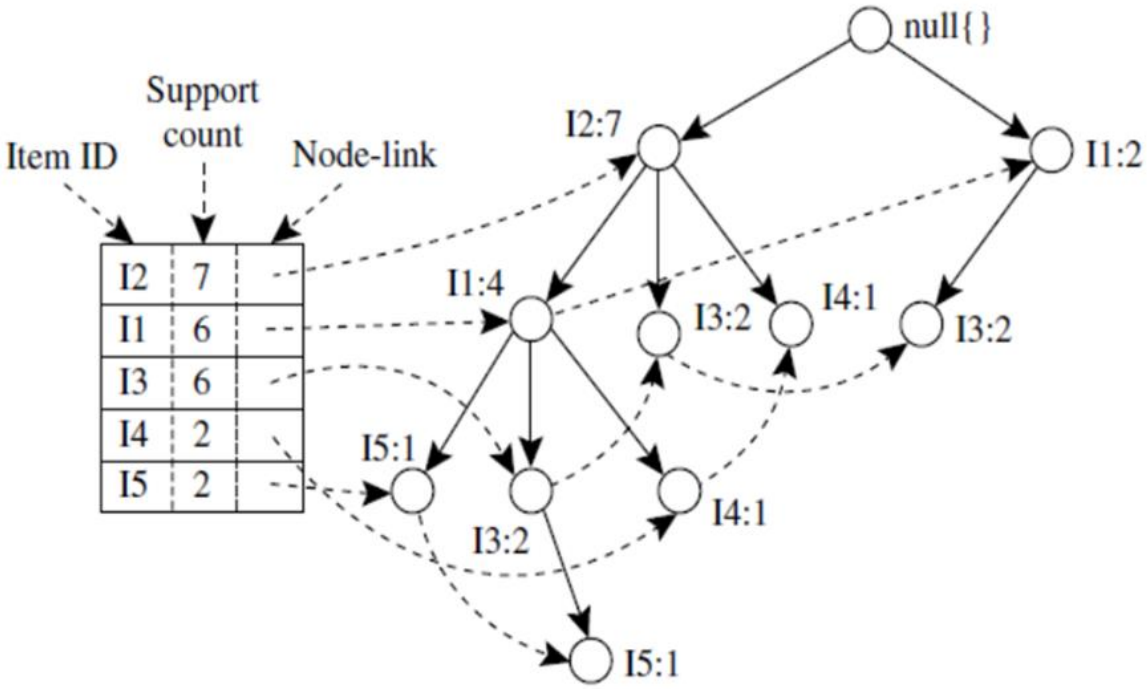
1. One root labeled as “null"

2. Each node in the item prefix sub-trees has three fields:

- **item-name**
- **count,** the number of transactions represented by the portion of the path reaching this node,
- **node-link** that links to the next node in the FP-tree carrying the same item-name.

3. Each entry in the frequent-item header table has two fields,

- **item-name**, and **head of node-link** that points to the first node in the FP-tree carrying the item-name.



An FP-tree registers compressed, frequent pattern information.

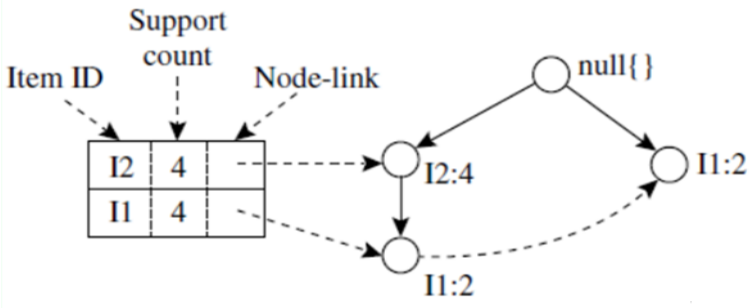
4.5.2 Frequent Itemset Generation in FP-Growth Algorithm

Mining Frequent Patterns Using FP-tree

- The FP-tree is mined as follows.
- Start from each frequent length-1 pattern (as an initial **suffix pattern**), construct its **conditional pattern base**
- then construct its (conditional) FP-tree, and perform mining recursively on the tree.
- The pattern growth is achieved by the concatenation of the suffix pattern with the frequent patterns generated from a conditional FP-tree.

Mining the FP-Tree by Creating Conditional (Sub-)Pattern Bases

Item	*Conditional Pattern Base*	*Conditional FP-tree*	*Frequent Patterns Generated*
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}	⟨I2: 2, I1: 2⟩	{I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2}
I4	{{I2, I1: 1}, {I2: 1}}	⟨I2: 2⟩	{I2, I4: 2}
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}	⟨I2: 4, I1: 2⟩, ⟨I1: 2⟩	{I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2}
I1	{{I2: 4}}	⟨I2: 4⟩	{I2, I1: 4}



Mining the FP-Tree by Creating Conditional (sub) pattern bases

Steps:

1. Start from each frequent length-1 pattern (as an initial suffix pattern).
2. Construct its conditional pattern base
3. Then, Construct its conditional FP-Tree & perform mining on such a tree.
4. The pattern growth is achieved by concatenation of the suffix pattern with the frequent patterns generated from a conditional FP-Tree.
5. The union of all frequent patterns gives the required frequent itemset.
6. Now, Lets start from I5. The I5 is involved in 2 branches namely {I2 I1 I5: 1} and {I2 I1 I3 I5: 1}.
7. Therefore considering I5 as suffix, its 2 corresponding prefix paths would be {I2 I1: 1} and {I2 I1 I3: 1}, which forms its conditional pattern base.
8. Out of these, Only I1 & I2 is selected in the conditional FP-Tree because I3 is not satisfying the minimum support count.

Algorithm: FP_growth. Mine frequent itemsets using an FP-tree by pattern fragment growth.

Input:

- D, a transaction database;
- *min_sup*, the minimum support count threshold.

Output: The complete set of frequent patterns.

Method:

1. The FP-tree is constructed in the following steps:
   (a) Scan the transaction database D once. Collect F, the set of frequent items, and their support counts. Sort F in support count descending order as L, the *list* of frequent items.
   (b) Create the root of an FP-tree, and label it as "null." For each transaction *Trans* in D do the following.
   Select and sort the frequent items in *Trans* according to the order of L. Let the sorted frequent item list in *Trans* be $[p|P]$, where p is the first element and P is the remaining list. Call **insert_tree**$([p|P], T)$, which is performed as follows. If T has a child N such that $N.item\text{-}name = p.item\text{-}name$, then increment N's count by 1; else create a new node N, and let its count be 1, its parent link be linked to T, and its node-link to the nodes with the same *item-name* via the node-link structure. If P is nonempty, call **insert_tree**(P, N) recursively.

2. The FP-tree is mined by calling **FP_growth**(*FP_tree*, *null*), which is implemented as follows.

```
procedure FP_growth(Tree, α)
(1)  if Tree contains a single path P then
(2)      for each combination (denoted as β) of the nodes in the path P
(3)          generate pattern β ∪ α with support_count = minimum support count of nodes in β;
(4)  else for each a_i in the header of Tree {
(5)      generate pattern β = a_i ∪ α with support_count = a_i.support_count;
(6)      construct β's conditional pattern base and then β's conditional FP_tree Tree_β;
(7)      if Tree_β ≠ ∅ then
(8)          call FP_growth(Tree_β, β); }
```

FP-growth algorithm for discovering frequent itemsets without candidate generation.

Multiple Choice Question (MCQ'S)

1. Which of the following best describes the goal of association analysis?
   a) Classification of data
   b) Prediction of numeric values
   c) Discovery of interesting relationships between items in large datasets
   d) Clustering of similar records
   Answer: c) Discovery of interesting relationships between items in large datasets

2. In association analysis, support refers to:
   a) The accuracy of a rule
   b) The proportion of transactions containing an itemset
   c) The number of rules generated
   d) The strength of correlation between items
   Answer: b) The proportion of transactions containing an itemset

3. Which of the following is true about frequent itemsets?
   a) They always contain only one item
   b) They occur rarely in transactions
   c) They satisfy a minimum support threshold
   d) They are generated without considering support
   Answer: c) They satisfy a minimum support threshold

4. The purpose of frequent itemset generation is to:
   a) Predict class labels
   b) Identify item combinations that occur frequently in transactions
   c) Perform clustering
   d) Optimize decision trees
   Answer: b) Identify item combinations that occur frequently in transactions

5. The Apriori principle states that:
   a) If an itemset is frequent, then all its subsets are frequent
   b) If an itemset is frequent, all its supersets must also be frequent
   c) Support is always equal to confidence
   d) Rules are generated before itemsets
   Answer: a) If an itemset is frequent, then all its subsets are frequent

6. Which of the following is NOT implied by the Apriori principle?
   a) If a subset is infrequent, the superset will also be infrequent
   b) Frequent itemsets can be pruned based on infrequent subsets
   c) All supersets of a frequent itemset must be frequent
   d) Reducing search space is possible by using subset properties
   Answer: c) All supersets of a frequent itemset must be frequent

7. In the Apriori algorithm, candidate itemsets are generated by:
   a) Splitting frequent itemsets into smaller sets
   b) Joining frequent (k-1)-itemsets to form k-itemsets
   c) Random selection from transactions
   d) Pruning frequent subsets
   Answer: b) Joining frequent (k-1)-itemsets to form k-itemsets

8. During frequent itemset generation in Apriori, pruning is used to:
   a) Increase candidate itemsets
   b) Remove itemsets with infrequent subsets
   c) Merge duplicate rules
   d) Increase support of itemsets
   Answer: b) Remove itemsets with infrequent subsets

9. Confidence of a rule measures:
   a) How often items occur together
   b) The probability that the consequent occurs given the antecedent
   c) The proportion of transactions containing an item
   d) The correlation between two datasets
   Answer: b) The probability that the consequent occurs given the antecedent

10. Confidence-based pruning helps in:
    a) Eliminating rules that do not meet a minimum confidence threshold
    b) Increasing the number of rules generated
    c) Finding closed itemsets
    d) Optimizing decision tree depth
    Answer: a) Eliminating rules that do not meet a minimum confidence threshold

11. Rule generation in the Apriori algorithm is performed after:
    a) Classification step
    b) Frequent itemset generation
    c) Clustering step
    d) Data cleaning step
    Answer: b) Frequent itemset generation

12. In Apriori rule generation, the consequent of a rule is:
    a) Always a single item or itemset derived from frequent itemsets
    b) Randomly selected from transactions
    c) Independent of frequent itemsets
    d) Not related to confidence
    Answer: a) Always a single item or itemset derived from frequent itemsets

13. Which of the following is true about maximal frequent itemsets?
    a) They have no frequent supersets
    b) They always contain all items
    c) They are identical to closed frequent itemsets
    d) They always satisfy maximum confidence
    Answer: a) They have no frequent supersets

14. Maximal frequent itemsets provide:
    a) All subsets of frequent itemsets
    b) A compact representation of frequent itemsets by excluding supersets
    c) No information about frequent itemsets
    d) Direct generation of rules
    Answer: b) A compact representation of frequent itemsets by excluding supersets

15. Closed frequent itemsets are defined as:
    a) Itemsets with no subsets
    b) Itemsets with identical support to their supersets
    c) Itemsets whose support is higher than confidence
    d) Rules generated from infrequent itemsets
    Answer: b) Itemsets with identical support to their supersets

16. A closed frequent itemset is always:
    a) A superset of all itemsets
    b) A subset with minimum support
    c) Unique in terms of its support count
    d) Generated only by FP-growth
    Answer: c) Unique in terms of its support count

17. FP-growth algorithm is preferred over Apriori because:
    a) It requires more candidate generation
    b) It avoids repeated database scans
    c) It does not use support
    d) It generates rules directly
    Answer: b) It avoids repeated database scans

18. The key idea of FP-growth is to:
    a) Use depth-first search only
    b) Store transactions in a compressed prefix-tree structure
    c) Perform clustering before itemset mining
    d) Eliminate the need for confidence pruning
    Answer: b) Store transactions in a compressed prefix-tree structure

19. An FP-tree represents:
    a) All itemsets in a dataset without compression
    b) Only maximal frequent itemsets
    c) The frequency of items in a compressed prefix-tree format
    d) Randomly selected transactions
    Answer: c) The frequency of items in a compressed prefix-tree format

20. In FP-tree construction, items in a transaction are:
    a) Inserted in increasing support order
    b) Inserted in decreasing support order
    c) Randomly ordered
    d) Not ordered at all
    Answer: b) Inserted in decreasing support order

21. FP-growth generates frequent itemsets by:
    a) Using recursive conditional FP-trees
    b) Generating all candidate itemsets
    c) Random pruning of itemsets
    d) Merging maximal itemsets only
    Answer: a) Using recursive conditional FP-trees

22. Which of the following is an advantage of FP-growth over Apriori?
    a) It requires fewer database scans and generates itemsets without candidate generation
    b) It always generates closed itemsets only
    c) It replaces support and confidence with clustering
    d) It can only mine maximal itemsets
    Answer: a) It requires fewer database scans and generates itemsets without candidate generation